Neuromorphic Computing and Engineering

PURPOSE-LED PUBLISHING™

PAPER • **OPEN ACCESS**

Training and synchronizing oscillator networks with Equilibrium Propagation

To cite this article: Théophile Rageau and Julie Grollier 2025 *Neuromorph. Comput. Eng.* **5** 034008

View the article online for updates and enhancements.

You may also like

- Reconstructing effective phase connectivity of oscillator networks from observations
Björn Kralemann, Arkady Pikovsky and Michael Rosenblum

- Fixed-time synchronization of kuramoto-oscillator networks with a simpler non-chattering controller
Yuchen Cai, Xiaoting Liu, Xiangxin Yin et al.

- Hyperbolic geometry of Kuramoto oscillator networks
Bolun Chen, Jan R Engelbrecht and Renato Mirollo

This content was downloaded from IP address 203.18.51.109 on 08/12/2025 at 12:08

IOP Publishing Neuromorph. Comput. Eng. 5 (2025) 034008 https://doi.org/10.1088/2634-4386/adebaa

NEUROMORPHIC
Computing and Engineering

CrossMark

OPEN ACCESS

RECEIVED
16 April 2025

REVISED
10 June 2025

ACCEPTED FOR PUBLICATION
3 July 2025

PUBLISHED
25 July 2025

Original Content from this work may be used under the terms of the Creative Commons Attribution 4.0 licence.

Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.

PAPER

Training and synchronizing oscillator networks with Equilibrium Propagation

Théophile Rageau* and **Julie Grollier**

Laboratoire Albert Fert, CNRS, Thales, Université Paris-Saclay, 1 avenue Augustin Fresnel, 91767 Palaiseau Cedex, France
* Author to whom any correspondence should be addressed.

E-mail: theophile.rageau@cnrs-thales.fr and julie.grollier@cnrs-thales.fr

Keywords: Equilibrium Propagation, synchronization, Kuramoto oscillators, neural networks

Abstract

Oscillator networks represent a promising technology for unconventional computing and artificial intelligence. Thus far, these systems have primarily been demonstrated in small-scale implementations, such as Ising machines for solving combinatorial problems and associative memories for image recognition, typically trained without state-of-the-art gradient-based algorithms. Scaling up oscillator-based systems requires advanced gradient-based training methods that also ensure robustness against frequency dispersion between individual oscillators. Here, we demonstrate through simulations that the Equilibrium Propagation algorithm enables effective gradient-based training of oscillator networks, facilitating synchronization even when initial oscillator frequencies are significantly dispersed. We specifically investigate two oscillator models: purely phase-coupled oscillators and oscillators coupled via both amplitude and phase interactions. Our results show that these oscillator networks can scale successfully to standard image recognition benchmarks, such as achieving nearly 98% test accuracy on the Modified National Institute of Standards and Technology (MNIST) dataset, despite noise introduced by imperfect synchronization. This work thus paves the way for practical hardware implementations of large-scale oscillator networks, such as those based on spintronic devices.

1. Introduction

Unconventional computing technologies are becoming essential for the new generation of artificial intelligence systems. Among these, oscillator networks are one of the most promising [1]. Various hardware implementations show that this technology can already be used to solve complex tasks such as combinatorial problems [2–8], visual pattern recognition [9–11] as well as to recognize spoken vowels with high precision [12, 13]. From a biological perspective, neurons in the brain behave like nonlinear oscillators [14] and can exhibit synchronization phenomena [15–17] in a way comparable to coupled oscillators. Networks of coupled oscillators have been applied to image recognition tasks through associative memory methods [18]. The principle of associative memory networks is to assign a local minimum of network energy to each image. This can be achieved by setting the local couplings between oscillators or using one single time-dependent coupling that interacts differently with each oscillator. The most common model of coupled oscillators used in associative memory networks is the Kuramoto model [19]. In the last decades, several approaches have been developed to extend this method to different kinds of oscillator networks (such as spin-torque [20] or ring oscillators [21]) and to increase its performances [22]. However, one remaining limitation of this approach is that it scales poorly to larger and more complex sets of images [9], especially compared to artificial neural networks with gradient-based approaches. Associative memory combined with gradient-based approaches seems to increase overall performance [21].

To train such a physical system (i.e. to adjust the weights of the connections between each pair of oscillator neurons), the standard Backpropagation algorithm [23] (or its variant for temporal tasks [24]) faces several challenges. The non-locality of its learning rule (derived by the chain rule) and the different types of neural computations involved (forward and backward passes implement different computations)

© 2025 The Author(s). Published by IOP Publishing Ltd

make its implementation at the hardware level very difficult [25]. In this context, several energy-based learning algorithms have been developed to train analog models (see [26] for a comparison on standard databases). In particular, the Equilibrium Propagation algorithm [27] is a learning framework that has the potential to solve the issues of the Backpropagation algorithm for on-chip training. It is a gradient-based learning framework, implementable at the hardware level, for a neural network with dynamic neurons. The algorithm alternates between relaxing the network to its minimum energy configuration and updating the network weights in the direction that minimizes a cost function. Compared to backpropagation, this algorithm provides a local learning rule to update the coupling weights, and its inference and learning phases involve the same type of computations. To date, physical implementations of the Equilibrium algorithm have been made with resistive networks [28, 29] and Ising machines deployed on the D-Wave platform [30]. Developments around this algorithm are still ongoing and have made it possible to get closer and closer to backpropagation's performance on large databases [31].

Coupled-phase oscillator network could be an alternative hardware to implement Equilibrium Propagation, as shown in some recent simulation results [32]. However, in the proposed framework, its implementation assumes that oscillators remain synchronized during training, which is rarely the case in physical oscillatory systems. For example, spin-torque nano-oscillators in a network can reach frequency dispersions (FDs) values 5% [33], which makes a full synchronization difficult to achieve in practice [34].

In this work, we adapt the Equilibrium Propagation algorithm to a network of coupled-phase oscillators with different natural frequencies. In particular, we show that Equilibrium Propagation enables training oscillatory systems while synchronizing oscillators. To demonstrate this property, we focus mainly on the Kuramoto model [19], which is a standard framework to describe the dynamics and synchronization of phase-coupled oscillators [35]. Then, we investigate the universal nonlinear oscillator model [36] that captures the coupled phase-amplitude dynamics of a wide range of nonlinear oscillators. Specifically, the contributions of the present work are the following:

- We achieve a test accuracy of 97.77% on the Modified National Institute of Standards and Technology (MNIST) database, representing the best known results using a network of fully synchronized Kuramoto phase-coupled oscillators trained with Equilibrium Propagation.
- We demonstrate analytically and with simulations that Equilibrium Propagation enables synchronization of a large population of oscillators during training.
- We show that the learning procedure of Equilibrium Propagation holds even with oscillator FDs of around 5%–10%. In that case, we are able to achieve test accuracies of 96%–97% on the MNIST database.
- We adapt the framework provided by the Equilibrium Propagation algorithm to the universal nonlinear oscillator model that considers coupled amplitude-phase oscillators. We show that this model can solve the MNIST database with a test accuracy of 96.85% using parameters representative of hardware spintronic systems.

This work is organized as follows: First we present the Kuramoto model that we used to describe the dynamics and synchronization of network of phase-coupled oscillators. Then, we adapt the learning framework given by the Equilibrium Propagation algorithm to the Kuramoto model and explain how synchronization occurs during training in this type of network. Finally, we present the results obtained with a network of phase-coupled oscillators trained with Equilibrium Propagation on the MNIST database.

2. General Kuramoto model

In this section, we present the general model of phase-coupled oscillators that we are using throughout this work.

2.1. Dynamics of phase-coupled oscillators

We consider a network of coupled-phase oscillators governed by the Kuramoto model as represented in figure 1(see figure 1(A) for the oscillator model and figure 1(B) for the network), where the phases of the connected oscillators continuously interact and influence each other. In this model, oscillators are described solely by their phases, with amplitudes assumed to be constant. Mathematically, the phase dynamics is given, in the stationary frame, by the system of coupled equations:

$$\frac{\mathrm{d}\phi_j}{\mathrm{d}t} + \omega_j = \sum_{k=1}^{K} \Omega_{j,k} \sin\left(\phi_k - \phi_j\right), \quad \forall j \in [\![1, K]\!] \tag{1}$$

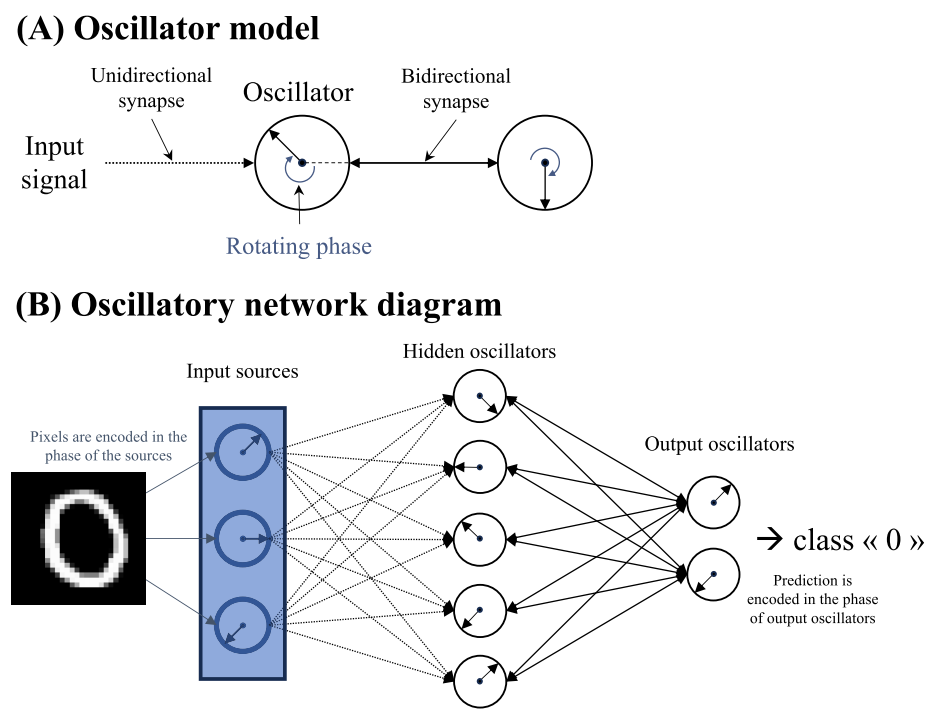

Figure 1. (A): Model of an oscillator in a network. The oscillator is represented by a vector in polar coordinates. In the Kuramoto model, the radius (amplitude of the oscillator) is fixed. The angle corresponds to its phase rotating at its frequency. The oscillator is driven by sources through unidirectional synapses (dotted line). It interacts with other oscillators using bidirectional synapses (solid line). (B): Diagram of a network of phase-coupled oscillators that follows the oscillator model in figure A. An input image is fed to the network. Its pixels are encoded in the phases of the input sources (in blue) that drive the hidden oscillators using unidirectional synapses (dotted lines). The hidden and output oscillators mutually influence their dynamics through bidirectional synapses (solid lines). Output oscillators encode the predicted label of the input image.

where:

$\phi = (\phi_j)_{(1 \leqslant j \leqslant K)}$: oscillator phases

$\omega = (\omega_j)_{(1 \leqslant j \leqslant K)}$: oscillators' natural frequencies

$\Omega = (\Omega_{j,k})_{(1 \leqslant j,k \leqslant K)}$: coupling matrix

In this equation, the oscillator phases operate at their natural frequencies ω_j, which may differ from one oscillator to another. The phase dynamics is influenced by coupling terms $\Omega_{j,k}$ which strengthen or reduce phase interactions between one oscillator and its neighbors.

2.2. Oscillator synchronization in a network

In some situations, oscillators in a network may synchronize partially or completely, in which case their phases start to operate at the same frequency, called synchronization frequency. More precisely, a set of coupled oscillators defines a locking range that depends on the strength of the interactions. If the natural frequency of an oscillator is inside this range, its instantaneous frequency will be locked to the synchronization frequency.

We illustrate the synchronization phenomena in figure 2. We simulate a fully-connected layered network (see appendix A.1 for more details on the network architecture) of coupled oscillators with 64 input sources, which emit at the same frequency ω_0, connected to 50 hidden oscillators, themselves connected to 10 output oscillators. The natural frequencies of the hidden and output oscillators are sampled in a Gaussian distribution around the common frequency of the sources ω_0. We vary the FD by tuning the standard deviation of the distribution. The coupling weights between the hidden oscillators, the input sources, and the output oscillators are randomly set (see appendix A.1 for the weight initialization). Figure 2 shows the evolution of the oscillator phases in the hidden layer for two networks with different FD. Phases are plotted in the rotating frame at the frequency of the sources ω_0. The first network (figure 2(A)) has no FD, whereas the other (figure 2(B)) has a FD of 0.5%. In appendix A.8 we provide more details on the simulation framework (physical considerations and technical details).

In figure 2(A), we observe that phases are first reconfigured and then, they all stabilize to a synchronized state at the frequency of the sources with fixed phase differences. The vertical dashed black line shows the transition between the reconfiguration regime and the stationary regime (from 600 nanoseconds). In figure 2(B), not all phases stabilize after some time. Some of them diverge. These oscillators are not

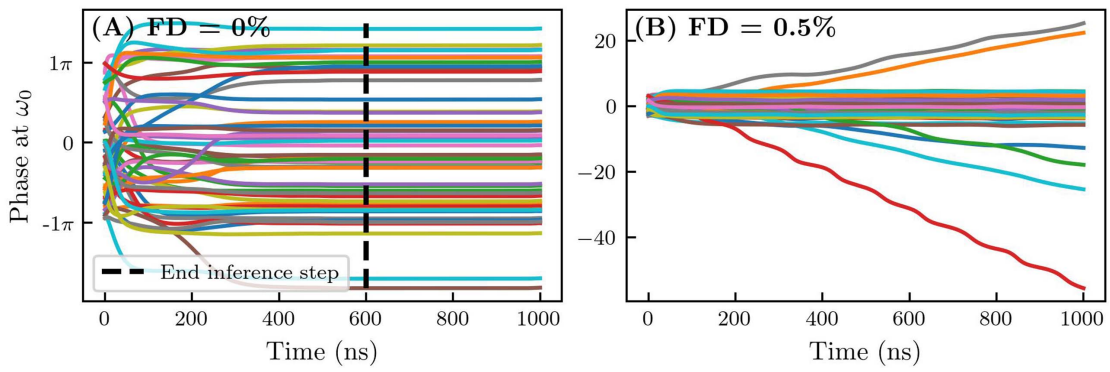

Figure 2. Phase evolution of hidden layer oscillators for two different frequency dispersions (FDs). The phases are observed in the rotating frame at frequency ω_0 (frequency of the input sources). Figure (A) shows the phase evolution of oscillators with no frequency dispersion, and (B) with a frequency dispersion of 0.5%. The vertical dashed black line on the left side shows the transition from a transient regime during which phases are reconfigured to a synchronization regime in which phases differences are fixed. This synchronization regime does not necessarily occur in the case of a network with frequency dispersion, as illustrated in figure (B) in which some phases do not settle to a stable configuration.

synchronized at the source frequency. Their natural frequency is outside the locking range of frequencies. However, most oscillators synchronize and act as in figure 2(A) but the non-synchronized oscillators disturb the stable configuration.

3. Training an oscillatory network with Equilibrium Propagation

In this section, we explain how we manage to train a network of phase-coupled oscillators using the Equilibrium Propagation algorithm. We derive the important equations of the training and investigate how synchronization occurs and evolves during training.

3.1. Adaptation of the Kuramoto model to the framework of Equilibrium Propagation

To train a network of phase-coupled oscillators with the Equilibrium Propagation algorithm, we need to describe its dynamics from an energy point of view, in order to deduce the learning rule of the model parameters, i.e. the coupling matrix $\Omega = (\Omega_{i,j})_{i,j\in[1,K]}$. The internal energy of the system E_ϕ derives from the dynamic equation (1). The time evolution of the system minimizes the energy:

$$\frac{\partial E_\phi}{\partial \phi_j} = -\frac{\mathrm{d}\phi_j}{\mathrm{d}t}. \tag{2}$$

Hence, the phase energy in the stationary frame is:

$$E_\phi := \sum_{j=1}^{K} \omega_j \phi_j - \sum_{j=1}^{K}\sum_{k=1}^{K} \Omega_{j,k} \cos\left(\phi_k - \phi_j\right). \tag{3}$$

Oscillators in the last layer of the network, called output oscillators (see figure 1(A) for example), are used as read-outs for the predictions made by the system. They aim to indicate the target that corresponds to a given input. The distance between the output units and the targets is measured by a cost function $\mathcal{L}$. The key idea of Equilibrium Propagation is to make this cost function act like a reconfigurable potential in the energy, that tends to drive the phases of the output oscillators towards a discriminative state. To control the strength of this potential, we introduce a scalar parameter β called the 'nudging factor' and define the total energy function (as described in [27]):

$$F_\phi\left(\phi, \Omega, \psi_{\text{target}}, \beta\right) := E_\phi\left(\phi, \Omega, \psi_{\text{target}}\right) + \beta \mathcal{L}\left(\phi, \psi_{\text{target}}\right). \tag{4}$$

With

$\phi = (\phi_j)_{(1\leqslant j\leqslant K)}$: oscillator phases
$\Omega = (\Omega_{j,k})_{(1\leqslant j,k\leqslant K)}$: learning parameters
$\psi_{\text{target}} = (\psi_{\text{target,o}})_{(1\leqslant o\leqslant O)}$: source phases that encode the target
β: nudge factor
$\mathcal{L}$: loss (or cost) function.

Different loss functions $\mathcal{L}$ can be considered as energy potentials. In [27] and [30], the authors proposed using the standard mean squared error with a network of continuous or binary Ising spins. For a similar system in [37], the cross-entropy loss function is introduced. Otherwise in [32], a network of synchronized oscillators is trained with a variant of the cross-entropy loss function adapted to the system. All these loss functions are designed to discriminate between good and bad predictions of the output neurons. In our approach, we choose a loss function intrinsically related to phase-coupled oscillator dynamics. We couple the output oscillators to sources (oscillators with fixed phases) that encode the target. The coupling between both sets of oscillators is set to the 'nudging factor' β:

$$\beta\mathcal{L}\left(\phi,\psi_{\text{target}}\right) := \sum_{o=1}^{O}\beta\cos\left(\psi_{\text{target,o}}-\phi_o\right).$$

The minimization of this loss function gives output phases equal to $\psi_{\text{target}}+\pi\,[2\pi]$. This state corresponds to the discrimination state that we want to achieve (please refer to appendix A.2 for more information on the encoding procedure). Depending on the ratio between the internal coupling matrix Ω and the external coupling β, the network relaxation will minimize the total energy F_ϕ by giving different importance to the two terms.

The Equilibrium Propagation algorithm has been extensively detailed in [27, 37, 38]. We give here a quick reminder on the global learning procedure focusing on important details for coupled oscillators (see appendix A.4 for the pseudo-code).

For each pair of input and target, the algorithm consists of two phases, referred to as the 'free' and 'nudge' phases. In the 'free' phase, we set the 'nudging factor' β to 0 and let the network relax to a stable configuration ϕ^0. The phase configuration achieved corresponds to a local minimum of the internal energy E_ϕ. In the 'nudge' phase, we set β to a non-zero value and then let the network converge to another stable configuration ϕ^β which corresponds to a local minimum of the total energy F_ϕ. This second local energy minimum is slightly different from the one obtained in the 'free' phase. It has been influenced by the external energy potential $\beta\mathcal{L}$ in the total energy F_ϕ, which means that the values of the output neurons are closer to the actual targets in ϕ^β than in ϕ^0. Then, the deviation between both stable configurations provides the gradient update of the learning parameter Ω:

$$\Delta\Omega \propto -\frac{1}{\beta}\left(\frac{\partial F_\phi}{\partial\Omega}\left(\phi^\beta,\Omega,\beta\right)-\frac{\partial F_\phi}{\partial\Omega}\left(\phi^0,\Omega,0\right)\right). \tag{5}$$

Finally, we write the new values of the learning parameters and iterate the whole procedure for another pair of input and target. In our oscillatory system, we derive the learning rule of the coupling parameters $\Omega_{j,k}$ from the general equation (5). If we denote respectively $\phi^0=(\phi_1^0,\ldots,\phi_K^0)$ and $\phi^\beta=(\phi_1^\beta,\ldots,\phi_K^\beta)$ the stable configurations of the phases after respectively the 'free' and 'nudge' phases, the following formula gives us the gradient updates for the parameter $\Omega_{j,k}$:

$$\Delta\Omega_{j,k} \propto \frac{1}{\beta}\left(\cos\left(\phi_k^\beta-\phi_j^\beta\right)-\cos\left(\phi_k^0-\phi_j^0\right)\right) \tag{6}$$

As described in [27], for vanishing values of β, these formulas approximate the gradient of the loss function $\nabla_\theta\mathcal{L}$. Hence, following this direction with a gradient-descent algorithm (such as stochastic-gradient-descent [39] or Adam algorithm [40]), we should be able to reach a minimum of $\mathcal{L}$ and solve the desired machine learning task.

3.2. Learning in a network of non-synchronized oscillators

A key assumption of Equilibrium Propagation is that it requires a system with converging units. It assumes that at the end of the relaxation ('free' and 'nudge' phases), the network settles in a stable configuration. However, as we saw in section 2.2, this requirement might not be fulfilled in an oscillatory network composed of oscillators with different natural frequencies. If the oscillators are synchronized at the same frequency, we can consider their phase in the rotating frame at the synchronization frequency and they can be seen as converging units, as shown in figure 2(A). If all oscillators are not synchronized, it is still possible to consider their phase in the same rotating frame, but the phases of non-synchronized oscillators will diverge, as shown in figure 2(B).

In practice, we observed that a subgroup of oscillators still synchronizes at the frequency of the sources. This can be explained by the fact that the frequencies of these oscillators are within the locking range. For these oscillators, it is therefore possible to define an internal coupling energy in the rotating frame of the sources (as given in equation (3)) and thus to deduce the learning rules.

For connections including a non-synchronized oscillator, phases do not rotate at the same frequency, and the requirements of Equilibrium Propagation are not met. However, it turns out that the learning rule used in the case of synchronized oscillators allows the initially non-synchronized oscillators to synchronize during training. In fact, when updating a connection between two oscillators, including one not synchronized, each cosine in the learning rule (6) will be a random value between -1 and 1 (because the phases do not evolve at the same frequency). Then, the update value $\Delta\Omega_{j,k}$ will also be a random number in the interval $[-\frac{2}{\beta}, \frac{2}{\beta}]$. This implies that the weights of the connections will evolve randomly if both connected oscillators are not synchronized at the same frequency. If, during this random walk, the weight of a connection encounters the synchronization region of the network, the oscillators will become synchronized, and the weight of the connection will then evolve according to the learning rule for synchronized oscillators. More details and examples on synchronizing oscillators during training can be found in appendix A.6.1.

Hence, the coupling changes during training modify both the synchronization state and the expressive power of the network. Therefore, there exists a trade-off between finding coupling weights that enable the synchronization of a large population of oscillators (to have the largest number of available neurons) and finding weights that enable minimizing the cost function (to solve the task).

3.3. Evolution of synchronization and accuracy during training

To illustrate how oscillator synchronization occurs during training, we focus on a small neural network (one hidden layer network with 50 hidden oscillators)trained with Equilibrium Propagation on a rescaled version of the MNIST database called Digits (https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html). Figure 3 shows in parallel the evolution of accuracy and the proportion of synchronized oscillators during training for two networks of Kuramoto oscillators with respectively a FD of 2% (figure 3(A)) and 5% (figure 3(B)). For each input image, we count the number of oscillators that synchronize at the frequency of the sources in the hidden layer of the network during the 'free' phase. Then, we averaged over all the training images for each epoch.

Several points can be highlighted from this figure. First, we observe that both networks can learn with different FDs and synchronization is also increasing during training. Both accuracy (red) and synchronization (blue) curves evolve in a very similar way. This indicates that oscillators globally manage to synchronize during training. As mentioned earlier, weight updates for non-synchronized oscillators are random, in contrast to the deterministic updates observed in synchronized ones. We find that these random updates tend to be significantly larger, allowing non-synchronized oscillators to undergo substantial frequency shifts that can eventually bring them into the synchronization range. A demonstration of this phenomenon using statistical tools is given in appendix A.6.2. A second observation from these curves is that, after a certain number of epochs, the system reaches a convergent state corresponding to an optimal balance found between network synchronization and its capacity of learning. Finally, for high-FD (figure 3(B)), there is a delay at the beginning of the training between the synchronization and accuracy curves. In fact, in the first few training epochs, the accuracy gets stuck to its initial value, whereas the synchronization increases. This is due to an initial intense reconfiguration of the network's couplings during the first training epochs. Indeed, at the beginning of training, the coupling values are random, so they do not favor synchronization within the network (the proportion of synchronized oscillators after the first epoch is around 30%). The iterative gradient updates of the couplings configure the network in a state that favors more oscillator synchronization (around 55% after 3 epochs). Once this new state is obtained, phases can be arranged to process information, which helps the network learn. This phenomenon appears very often while training a network with a high initial FD (from 4% to 5%). For a smaller FD (figure 3(A)), it is smaller or is even not apparent.

4. Performance on MNIST database

In this section, we investigate the performance of oscillatory networks on the standard MNIST database [41] in three different configurations: fully-synchronized Kuramoto model, general Kuramoto model and coupled amplitude-phase model.

The MNIST database, composed of 70 000 28×28 greyscale images of handwritten digits, from 0 to 9, including 60 000 images for the train set and 10 000 for the test set, is a standard classification task for testing machine learning models. Previous results have shown that a network of fully-synchronized coupled-phase oscillators can be trained with Equilibrium Propagation on a rescaled version of MNIST (8×8 images) comprising less than 2000 digit images [32].

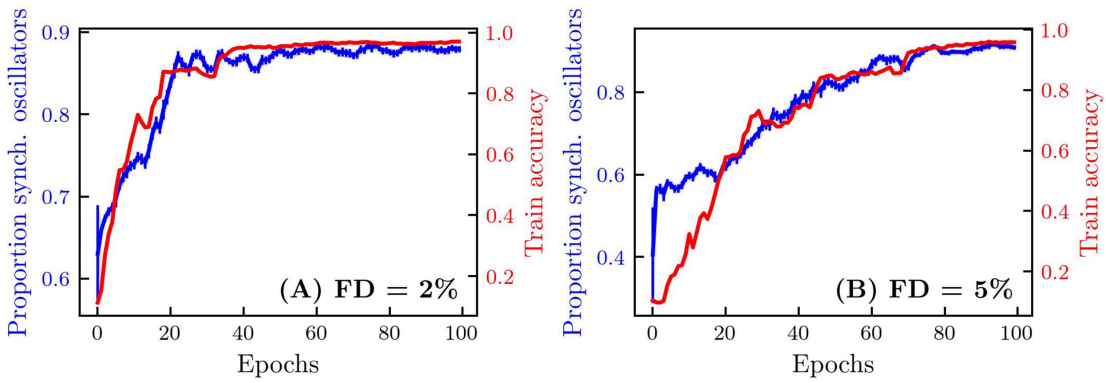

Figure 3. Evolution of accuracy and proportion of synchronized oscillators during training. We trained two networks of Kuramoto oscillators with a respective initial frequency dispersion of 2% (A) and 5% (B) on a small image database. In each figure, the blue curve shows the average proportion of synchronized oscillators over the training period (with standard deviations). The red curve shows in parallel the evolution of the accuracy of the model's learned data during training.

Table 1. Comparison of performance (test accuracy) for standard Ising networks (left) and the oscillatory Kuramoto network (right) on MNIST database.

Ising networks		Kuramoto network
Scellier and Bengio [27]	Ernoult *et al* [31]	This work
∼97.5%	97.94% ± 0.16%	97.77% ± 0.08%

4.1. Fully-synchronized Kuramoto model

The first configuration we investigate is a particular case of the general Kuramoto model described in section 2.1. Here, we assume that all oscillators share the same natural frequency ω_0, which is also the frequency of the sources. Then, the oscillators are, by definition, synchronized at their mutual frequency ω_0. This particular case removes all synchronization issues described in section 3.2. To tackle the full MNIST database, we use a fully-connected-layered network with one hidden layer of 500 neurons (see appendix A.1 for a detailed description of the fully-connected architecture). We also add external bias sources at frequency ω_0 to oscillators as proposed in [32]. This increases the number of learning parameters in our system. In appendix A.9, we discuss their impact on the network performance. The total size of our network is 784-500-10 ($28 \times 28 = 784$ pixels as input, 500 hidden neurons, and 10 output classes). We look for optimal values of the hyper-parameters. More details can be found in appendix A.8.5.

The comparison of the results is summarized in table 1. Both columns on the left show benchmark results obtained by training a standard Ising model with Equilibrium Propagation, described by a different energy function from the one we use here. With this model, [27] obtains an accuracy of 97.5% and [38] reaches 97.94% by enhancing the precision used to simulate the dynamics of the system. With our system, we reach an average test accuracy of 97.77% ± 0.08% (mean over 5 runs) on the MNIST database as shown in the right column of table 1. Therefore, our model compares very well with benchmark results for an equivalent Ising network trained with Equilibrium Propagation.

4.2. General Kuramoto model

In the previous section, we made the assumption that all oscillators have exactly the same natural frequency ω_0. However, this is rarely achieved in real physical systems. This is mostly due to the fact that the natural frequency of a device strongly depends on its fabrication process, yielding variability between devices on the same substrate. For several types of oscillators such as spin-torque nano-oscillators, FDs in a network may reach values above 5% [33].

Here, we simulate a network of Kuramoto oscillators with different natural frequencies, sampled in a Gaussian distribution around the frequency of the sources ω_0, and train it on the MNIST database. We vary the FD by tuning the standard deviation of the Gaussian distribution. The FD measures the rate of displacement compared to the mean frequency. The standard deviation is then equal to the FD times the mean frequency.

Figure 4 shows the evolution of the accuracy on MNIST for each FD. We observe that the performance of our model deteriorates as FD increases. However, this reduction is moderate and we still achieve more than 95% accuracy on the test set with high FDs (10%–15%). The key to maintaining high performance even with

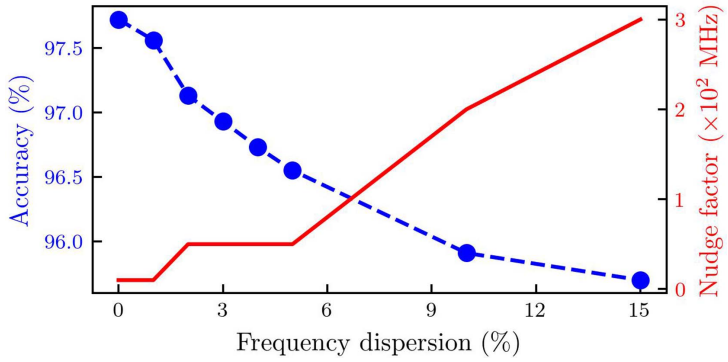

Figure 4. Evolution of accuracy and 'nudge factor' (in hundreds of MHz) as a function of frequency dispersion around the mean frequency $\omega_0 = 4.2$GHz. For each value of the frequency dispersion, we trained a network on the MNIST database and recorded the best accuracy obtained on the test set. The blue curve shows the evolution of the accuracy as the frequency dispersion within the oscillator network increases. The red curve shows in parallel the evolution of the 'nudge factor' we had to use to obtain these results.

large dispersions is to properly choose the 'nudge factor' β. In our simulations, we observed that for high FDs, small values of β led to poor quality learning while increased values of the nudge factor lead to improved results. This phenomena can be explained by the internal dynamics of our network. Indeed, couplings between oscillators tend to synchronize them at a certain frequency of the sources ω_0. However a few oscillators may rarely or never synchronize to the others if their natural frequency is too distant from the average frequency of the network. These few oscillators still influence the dynamics of the network and act like background noise on the output oscillator phases. Then, by using a small 'nudging factor', the network cannot differentiate between an actual nudging term and noise. Thus, the greater the number of non-synchronized oscillators, the larger the required 'nudging factor'.

These results show that training a network of Kuramoto phase-coupled oscillators with Equilibrium Propagation is robust to frequency variability. This opens up perspective for future hardware implementation, in which frequency variability can be a strong issue.

4.3. Fully-synchronized coupled amplitude-phase model

In this section, we look at a more general model of coupled oscillators. Whereas the Kuramoto model assumes fixed oscillator amplitudes, we consider here that the oscillators are instead characterized by an amplitude and a phase that operate in a coupled way. For this purpose, we use the universal nonlinear oscillator model with negative damping and nonlinear frequency shift proposed in [36]. This model is very general and can be used to describe different types of auto-oscillatory systems.

Following the theory described in [36], the dynamics of a system of K coupled nonlinear oscillators is described by the following coupled equations for powers p (amplitude squared) and phases ϕ (the derivation of both equations can be find in appendix A.7.2):

$$\begin{aligned}
&\frac{\mathrm{d}p_j}{\mathrm{d}t} + 2p_j\left(\Gamma_+\left(p_j\right) - \Gamma_-\left(p_j\right)\right) = 2\sum_{k=1}^{K}\Omega_{k,j}\sqrt{p_j p_k}\cos\left(\phi_k - \phi_j + \beta_{k,j}\right)\\
&\frac{\mathrm{d}\phi_j}{\mathrm{d}t} + \omega_j\left(p_j\right) = \sum_{k=1}^{K}\Omega_{k,j}\sqrt{\frac{p_k}{p_j}}\sin\left(\phi_k - \phi_j + \beta_{k,j}\right)
\end{aligned} \tag{7}$$

where:

$p = (p_j)_{(1\leqslant j\leqslant K)}$: oscillator powers in the network
$\phi = (\phi_j)_{(1\leqslant j\leqslant K)}$: oscillator phases in the network
$\omega_j(p) = \omega_{0,j} + Np$: frequency of oscillator j
$\Gamma_+(p) = \Gamma_G(1 + Qp)$: positive damping
$\Gamma_-(p) = \sigma I(1 - p)$: negative damping
$\Omega = (\Omega_{j,k})_{(1\leqslant j,k\leqslant K)}$: coupling matrix
$\beta_{k,j}$: coupling phase between oscillators k and j

In the phase equation, several terms are similar to those that appear in the Kuramoto dynamics (see equation (1)) such as pairwise couplings $\Omega_{i,j}$ and the frequency term ω_j (more information on this model can be found in appendix A.7.4).

From this dynamics, we adapt this oscillator model to the Equilibrium Propagation framework, and derive the learning rule of the coupling matrix Ω (see derivations and results in appendix A.7.2).

We tested the performance of this model on the MNIST database. To be able to compare the results with those obtained by the Kuramoto model, we use the same network architecture as in the previous cases (i.e. fully-connected network with one hidden layer of size 784-500-10 and external bias sources). We also assumed that oscillators and bias in the network all have the same natural frequency ω_0.

With the coupled amplitude-phase model of oscillator, we were able to obtain $96.85\% \pm 0.16\%$ accuracy on the MNIST database. More information on the physical considerations and technical details (optimization, computational power and simulations issues) are given in appendix A.8.

One particular observation we made during the fine-tuning process was that this kind of network never reached 100% accuracy on the training set, which is usually the case when training deep learning models. Moreover, this phenomenon occurred regardless of the database considered (smaller databases such as 2D databases, or larger one as MNIST). For each trial, there was a gap of around 1% between perfect precision and precision on the training set. Taking this into account, the result for the test set appears to be more consistent, with a generalization gap of around 2% between the training set and the test set. We observed the same kind of generalization gap for the Kuramoto model.

The results obtained are about 1% less than those obtained with the Kuramoto model. As the Kuramoto model directly derives from the nonlinear universal model, we might have expected similar accuracies for comparable network architectures and hyperparameters (see table A1 in appendix for a comparison). We attribute the difference to the fact that we use an approximation of the energy function to derive the learning rules of the model that may reduce the precision of weight updates in the last stage of training (see appendix A.7.2 for more information).

5. Conclusion

In this work, we have shown through simulations that a network of phase-coupled oscillators can be a candidate device for a hardware implementation of the Equilibrium Propagation algorithm. We demonstrated that Equilibrium Propagation intrinsically enables synchronization in oscillatory networks through a random walk of weights connecting non-synchronized oscillators. The results presented in table 1 show that a network of phase-coupled Kuramoto oscillators is capable of obtaining results comparable to those of the state of the art with an Ising model of spins on the MNIST database. We have also shown that training a network of this type is relatively robust to frequency variability, which is common in certain oscillator models. Finally, we looked at the spintronic model of coupled oscillators with phase and amplitude and showed that it is possible to train it with a 1% accuracy loss on the MNIST database.

Data availability statement

The data that support the findings of this study are openly available at the following URL/DOI: https://github.com/neurophysics-cnrsthales/training-oscillatory-net-EqProp.

Acknowledgment

This project was supported with AI computing and storage resources by GENCI at IDRIS thanks to Grant 2023-AD010913993 on the supercomputer Jean Zay's V100 partition.

Conflict of interest statement

The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Author contributions

The project was designed by J G and T R. The code and numerical simulations were all carried out by T R. T R and J G wrote the manuscript.

Funding

This work was funded by the European Research Council advanced grant GrenaDyn (reference: 10 102 0684) and by a public grant overseen by the French National Research Agency (ANR) as part of the 'PEPR IA France 2030' program (Emergences project ANR-23-PEIA-0002)

Appendix

A.1. Network architecture and initialization

In this work, all the conducted simulations were done using fully-connected-layered networks with one hidden layer (also called one-hidden layer networks), composed of an input layer, one hidden layer, and an output layer. We decided to focus on this specific network architecture, mostly inspired from deep learning works, instead of more 'physically-based' architectures such as all-to-all or nearest-neighbors couplings. In order to develop a hardware implementation of a neuromorphic system, these 'physically-based' architectures would need to be deeply investigate. However, the work of this article aims to present oscillatory networks as a candidate hardware for future neuromorphic applications. In that sense, to be able to benchmark our system with those already existing [27, 32, 38], we decided to use a comparable network architecture.

To train our oscillatory network, we need to encode the samples of the database we want to learn in the phase domain (see appendix A.2 for more details). In this work, a sample usually corresponds to an input image (grid of pixels) and a class label. To encode this kind of sample, we use oscillatory sources, i.e. oscillators for which we can tune their phase. For an input image, each source encodes one pixel. Thus, an $N \times N$ image will require N^2 sources to be feed to the network. These input nodes are connected to the oscillators in the hidden layer. It corresponds to the features produced through network learning, and captures the underlying structures of the input data as well as the expressive power of the network. Then, oscillators in the hidden layer are all connected to output oscillators that encode the prediction of the network (the label of an input image). The whole architecture of the network is depicted in figure 1. The input sources are fully-connected to hidden oscillators and continuously drive their dynamics. Hidden and output oscillators are fully-connected in a bidirectional way and therefore, they influence each other's dynamics.

The weights between two consecutive layers l and $l+1$ of a network are initialized using a uniform law $\mathcal{U}(\frac{-1}{\sqrt{N_l}}, \frac{1}{\sqrt{N_l}})$, where N_l is the size of the layer l. When present, biases are initialized as follows: the amplitude of the bias of the layer l is initialized also with $\mathcal{U}(\frac{-1}{\sqrt{N_l}})$ and its phase is initialized as a random number in the interval $[-\pi, \pi]$.

A.2. Data encoding in the phase domain

Both input (pixels of an image) and target (label of an input image) data are encoded in the phases of sources. These sources are connected to the network and will perturb its dynamics. Through our simulations, we observed that the encoding procedure of the data may have a significant impact on the system behavior, and then, on its performance once trained with Equilibrium Propagation.

We choose the following encoding procedure for the input and target data:

- Input: pixels of an input image are normalized in $[-\frac{\pi}{2}, \frac{\pi}{2}]$ and phases of input sources are set to the value of the transformed pixels.
- Target: target labels are vectorized using one-hot encoding, and modified by the linear function $f: c \to \frac{\pi}{2}c + \frac{\pi}{2}$. Then, phases of target sources are set to the value of the transformed labels.

Both these encoding procedures have been obtained and confirmed empirically. They have been shown to give the best results and we have no theoretical proof of their optimality. However, we can give some intuitions on that. Regarding the input encoding, we know that during its relaxation, the network tends to minimize, or maximize depending on the sign of the corresponding weight, the quantity $\cos(\phi_i^{\text{in}} - \phi_h^{\text{hid}})$, i.e. make this quantity close to -1 or 1. Then, it comes that $\phi_h^{\text{hid}} = \phi_i^{\text{in}} - \pi \quad [2\pi]$ or $\phi_h^{\text{hid}} = \phi_i^{\text{in}} [2\pi]$. Hence, by choosing the previous encoding procedure, it appears that hidden phases ϕ_h^{hid} will converge either in $[\frac{\pi}{2}, \frac{3\pi}{2}]$ or in $[-\frac{\pi}{2}, \frac{\pi}{2}]$ depending on the sign of the weight. The reunion of both intervals corresponds to the entire trigonometric circle with no overlap. If we had chosen an interval of greater length, there would have been an overlap between both intervals. If we had chosen an interval of smaller length, there would have been no overlap but the trigonometric circle would not have been covered completely. This implies that with our encoding procedure, we know the sign of the coupling weight from the phase value of the hidden oscillators. It introduces some structure inside the network that can be used during training.

Regarding the target encoding, the result was also obtained empirically. One interesting thing that we notice is that generally the network's performance are better when the phase difference in the encoding procedure is equal to $\frac{\pi}{2}$ rather than to π.

A.3. Choice of the loss function

In the system of Kuramoto oscillators, we use a loss function that is quite related to the dynamics of coupled oscillators, especially compared to common loss functions that are widely used in ,machine learning (mean squared error and cross entropy). This choice has been motivated mostly by the potential hardware implementation of the loss function. In fact, in Equilibrium Propagation, the loss function is used as a drive applied on the output oscillators to 'nudge' them towards a target. In that case, a natural way to drive the output oscillators is to connect them to sources at the same frequency with particular phases and let the phase interaction between oscillators and sources drive the phase of the output oscillators. In our simulations, we also tested the other mentioned loss functions, even if their potential hardware implementation in a network of coupled oscillators is not straightforward. For the mean square error, we observed results comparable to those obtained with the loss function based on Kuramoto dynamics. For the cross-entropy, we observed that this loss function is not symmetric, so we cannot use the negative 'nudge' in Equilibrium Propagation as proposed in [37]. This leads to a degradation of the gradient approximation. Based on these observations, we decided to use the loss function based on the Kuramoto dynamics instead of a more standard one.

A.4. Pseudo-code for Equilibrium Propagation

Below we give the pseudo code of the Equilibrium Propagation algorithm with three phases (prediction, positive nudge and negative nudge):

Algorithm 1. Equilibrium Propagation with three phases.

Inputs: X_{train}, Model, β, lr, max_epoch
for epoch $n \in$ [1,max_epoch] **do**
  for each image, label $(x, l) \in X_{train}$ **do**
    Encode image: $x_{enc} \leftarrow$ encode_image(x)
    Encode label: $l_{enc} \leftarrow$ encode_label(l)
    Initialize neuron units: $\Phi \leftarrow$ initialize() ▷Neuron initialization
    Free phase: $\Phi^0 \leftarrow \text{Model}(x_{enc}, \Phi)$ ▷Compute prediction
    Positive nudge phase: $\Phi^{\beta} \leftarrow \text{Model}(x_{enc}, \Phi^0, \beta, l_{enc})$
    Negative nudge phase: $\Phi^{-\beta} \leftarrow \text{Model}(x_{enc}, \Phi^0, -\beta, l_{enc})$
    Compute gradients: $\Delta\Omega \leftarrow$ equation (5)
    Update weights: $\Omega \leftarrow \Omega + lr \cdot \Delta\Omega$
  end for
end for

A.5. Details of equations for a one hidden layer Kuramoto network

In this section, we give details of the oscillator dynamics equations, the derivation of the network energy and the computation of the learning rules for the different learning parameters. The calculations are done for a network of Kuramoto oscillators with one hidden layer and external bias. These external signals are tunable, so they are new learning parameters in our model. All oscillators are assumed to share the same natural frequency ω_0. Then, we consider the equations in the rotating frame at frequency ω_0. So, we define and use the 'slow phase' (phase in the rotating frame) of an oscillator $\Phi = \phi + \omega_0$ instead of the phase in the stationary frame ϕ.

A.5.1. Equations of dynamics

$$\frac{\mathrm{d}\Phi_h^{\text{hid}}}{\mathrm{d}t} = \sum_{i=1}^{I} \Omega_{h,i}^0 \sin\left(\Phi_i^{\text{inp}} - \Phi_h^{\text{hid}}\right) + \sum_{o=1}^{O} \Omega_{o,h}^1 \sin\left(\Phi_o^{\text{out}} - \Phi_h^{\text{hid}}\right) + F_h^0 \sin\left(\Psi_h^0 - \Phi_h\right), \quad \forall h \in [\![1, H]\!]$$

$$\frac{\mathrm{d}\Phi_o^{\text{out}}}{\mathrm{d}t} = \sum_{h=1}^{H} \Omega_{o,h}^1 \sin\left(\Phi_h^{\text{hid}} - \Phi_o^{\text{out}}\right) + F_o^1 \sin\left(\Psi_o^1 - \Phi_o^{\text{out}}\right), \quad \forall o \in [\![1, O]\!]$$

where:

(I, H, O): number of oscillators (or sources) in each layer, respectively input, hidden and output

$((\Phi_i^{\text{inp}})_{(1\leqslant i\leqslant I)}, (\Phi_h^{\text{hid}})_{(1\leqslant h\leqslant H)}, (\Phi_o^{\text{out}})_{(1\leqslant o\leqslant O)})$: oscillator 'slow phases' at source frequency (sources for the input), respectively in the input, hidden and output layers

$\Omega^0 = (\Omega_{h,i})_{(1\leqslant h\leqslant H),(1\leqslant i\leqslant I)}$: coupling matrix between input and hidden layers

$\Omega^1 = (\Omega_{o,h})_{(1\leqslant o\leqslant O),(1\leqslant h\leqslant H)}$: coupling matrix between hidden and output layers

$(F^0, \Psi^0) = ({F^0}_h, \Psi_h^0)_{(1\leqslant h\leqslant H)}$: external drives (amplitude and phase) applied to oscillators in the hidden layer

$(F^1, \Psi^1) = ({F^1}_o, \Psi_o^1)_{(1\leqslant o\leqslant O)}$: external drives (amplitude and phase) applied to oscillators in the output layer

A.5.2. Derivation of the energy function

From the previous dynamics, we derive the energy function of a Kuramoto network E_Φ and the total energy F_Φ influenced by 'nudging' factor β. The dynamics of the oscillators that compose the network tend to minimize the energy of the system:

$$\frac{\partial E_\Phi}{\partial \Phi_h^{\text{inp}}} = -\frac{\mathrm{d}\Phi_h^{\text{inp}}}{\mathrm{d}t}, \quad \forall h \in [\![1, H]\!]$$
$$\frac{\partial E_\Phi}{\partial \Phi_o^{\text{out}}} = -\frac{\mathrm{d}\Phi_o^{\text{out}}}{\mathrm{d}t}, \quad \forall o \in [\![1, O]\!].$$

Hence,

$$E_\Phi := -\sum_{h=1}^{H}\sum_{i=1}^{I}\Omega_{h,i}^0 \cos\left(\Phi_i^{\text{inp}} - \Phi_h^{\text{hid}}\right) - \sum_{h=1}^{H}\sum_{o=1}^{O}\Omega_{o,h}^1 \cos\left(\Phi_o^{\text{out}} - \Phi_h^{\text{hid}}\right)$$
$$-\sum_{h=1}^{H} F_h^0 \cos\left(\Psi_h^0 - \Phi_h^{\text{hid}}\right) - \sum_{o=1}^{O} F_o^1 \cos\left(\Psi_o^1 - \Phi_o^{\text{out}}\right).$$

As described in section 3.1, during the nudge phase, we drive the output oscillators Φ_o^{out} towards target sources Ψ_{target}. The parameter β controls the strength of the drive interaction. We define the total energy F_Φ of the network:

$$F_\Phi\left(\Phi, \theta, \Psi_{\text{target}}, \beta\right) := E_\Phi\left(\Phi, \theta, \Psi_{\text{target}}\right) + \beta\mathcal{L}\left(\Phi, \Psi_{\text{target}}\right)$$

where

$\theta = (\Omega^0, \Omega^1, F^0, F^1, \Psi^0, \Psi^1)$: learning parameters

$\psi_{\text{target}} = (\psi_{\text{target,o}})_{(1\leqslant o\leqslant O)}$: source phases that encode target data

β: 'nudge factor'

$\mathcal{L}$: Loss function

A.5.3. Derivation of the learning rule

To obtain the learning rules, we use the formula proposed in [27] that relies on the total energy of the network F_ϕ and the steady states of the network after the 'free' and 'nudge' phases, respectively Φ^0 and Φ^β:

$$\Delta\theta_i \propto -\frac{1}{\beta}\left(\frac{\partial F_\Phi}{\partial \theta_i}\left(\Phi^\beta, \theta, \Psi_{\text{target}}, \beta\right) - \frac{\partial F_\Phi}{\partial \theta_i}\left(\Phi^0, \theta, \Psi_{\text{target}}, 0\right)\right).$$

Then, we obtain the following learning rules:

$$\Delta\Omega_{j,k}^0 \propto \frac{1}{\beta}\left(\cos\left(\Phi_k^{\text{inp},\beta} - \Phi_j^{\text{hid},\beta}\right) - \cos\left(\Phi_k^{\text{inp},0} - \Phi_j^{\text{hid},0}\right)\right)$$
$$\Delta\Omega_{j,k}^1 \propto \frac{1}{\beta}\left(\cos\left(\Phi_k^{\text{out},\beta} - \Phi_j^{\text{hid},\beta}\right) - \cos\left(\Phi_k^{\text{out},0} - \Phi_j^{\text{hid},0}\right)\right)$$

$$\Delta F_j^0 \propto \frac{1}{\beta}\left(\cos\left(\Psi_j^0 - \Phi_j^{\text{hid},\beta}\right) - \cos\left(\Psi_j^0 - \Phi_j^{\text{hid},0}\right)\right)$$
$$\Delta F_j^1 \propto \frac{1}{\beta}\left(\cos\left(\Psi_j^1 - \Phi_j^{\text{out},\beta}\right) - \cos\left(\Psi_j^1 - \Phi_j^{\text{out},0}\right)\right).$$

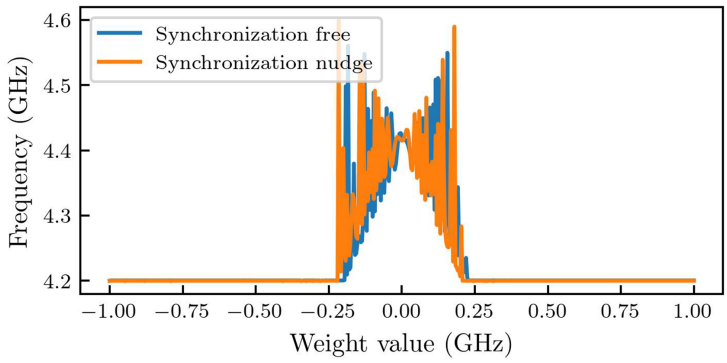

Figure A1. Evolution of the instantaneous frequency of the oscillator as a function of one coupling in a network of Kuramoto oscillators for the 'free' and 'nudge' phases of the Equilibrium Propagation algorithm.

$$\Delta\Psi_j^0 \propto \frac{F_j^0}{\beta}\left(\sin\left(\Psi_j^0 - \Phi_j^{\text{hid},0}\right) - \sin\left(\Psi_j^0 - \Phi_j^{\text{hid},\beta}\right)\right)$$
$$\Delta\Psi_j^1 \propto \frac{F_j^1}{\beta}\left(\sin\left(\Psi_j^1 - \Phi_j^{\text{out},0}\right) - \sin\left(\Psi_j^1 - \Phi_j^{\text{out},\beta}\right)\right)$$

A.6. Synchronization during training

In this section, we give details and examples of the phenomenon of oscillator synchronization during training with Equilibrium Propagation, as described in section 2.1. We first explain this phenomenon with simulation results and then give an analytical demonstration using statistical tools.

A.6.1. Empirical analysis of synchronization

To investigate oscillator synchronization during training, we simulate a network of Kuramoto oscillators and observe the behavior of one single oscillator. We set all oscillator frequencies at the same value ω_0, except for one oscillator whose frequency is set to ω_s (5% higher than ω_0) so that this oscillator is not initially synchronized with the others. Then, we feed the network with one input image (taken from the Digits database) and train only one coupling weight, between the chosen oscillator and a source. Finally, we observe the evolution of the considered weight during the training and how it affects the synchronization of the oscillator.

Figure A1 shows the evolution of the oscillator frequency over the coupling weight for the 'free' and 'nudge' steps of the Equilibrium Propagation framework. In this figure, we can see that if the weight is large enough (in absolute value), the frequency of the oscillator is equal to ω_0 i.e. the oscillator synchronizes to other oscillators. Between two thresholds values, the frequency of the oscillator is not equal anymore to ω_0, which means that the oscillator cannot synchronize. We distinguish the 'free' and 'nudge' steps of the Equilibrium Propagation framework because the network is not driven by the same elements in each case, hence it slightly changes its synchronization region.

Figure A2 shows on the left the evolution of the weight during training, and on the right the value of each weight update. In both of these plots, we highlight the synchronization state of the oscillator for each step of the Equilibrium Propagation framework ('free' and 'nudge'), and for both. Dashed black lines on the right show the maximum range of update that can be reached. On the left, we observe that the value of the weight is evolving in a random way until it reaches a value that enables the synchronization of the oscillator. Once the oscillator reaches this synchronization state, the value of the weight stabilizes and starts a slow convergence. On the right, we can see that first the weight updates seem to follow a random walk on the interval $[-\frac{2}{\beta}, \frac{2}{\beta}]$. Then, once the oscillator reaches a synchronized state, we observe that weight updates change radically their behavior and start a slow convergence towards 0. This case corresponds to the learning of the optimal weight regarding the initial task. Another interesting point to notice is that the amplitude of the weight update also changes radically between the non-synchronized and synchronized states. In fact, as shown in the histogram figure A3, before synchronization, the update follows a random path and covers the entire range of possible values. Then, when the oscillator reaches synchronization, the amplitude of the weight update decreases and starts converging towards 0. Hence, on the one hand, if two oscillators are not synchronized, the coupling between them will evolve randomly with a large amplitude, so it will relatively quickly reach the synchronization region. On the other hand, once the coupling is in the synchronization

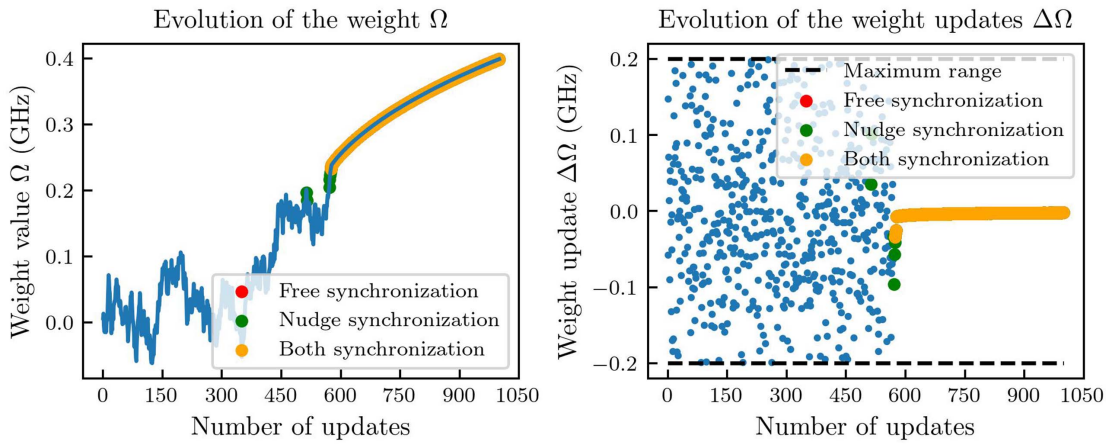

Figure A2. Evolution of the coupling weight (left) and weight update computed by the learning rule 6 (right) as a function of the number of updates for a single weight in a one hidden layer network with 50 hidden oscillators. Using different colors, we recorded the different synchronization states of the oscillator. The color blue means that the considered oscillator is not synchronized at the frequency ω_0. The color red means synchronization only during the 'free' phase. The color green means synchronization only during the 'nudge' phase. Finally, the color orange means synchronization during both 'free' and 'nudge' phases. We used $\beta = 0.1$ GHz and $lr = 0.12$.

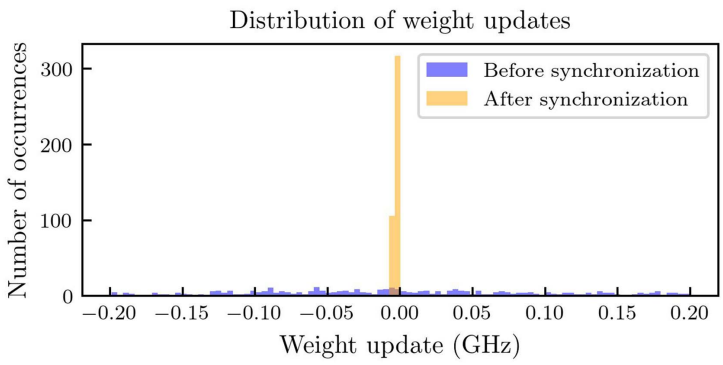

Figure A3. Histogram of the amplitude of the weight updates during training for one single oscillator in a one hidden layer network with 50 hidden oscillators. We split the weight updates using two different colors, depending on the synchronization state of the considered oscillator. The color blue corresponds to weight updates before the oscillator reaches a synchronization state (synchronization at ω_0 for both 'free' and 'nudge' phases), whereas the orange color corresponds to the weight updates after the oscillator reaches the synchronization state. We used $\beta = 0.1$ GHz and $lr = 0.12$.

region, its update amplitude will be smaller and deterministic, and so it has more chance to remain in the synchronization region. This fact explains why oscillators globally synchronize during training with Equilibrium Propagation.

A.6.2. Statistical analysis of synchronization

Using some statistical tools, we found that both cosine terms $\cos(\phi_k - \phi_j)$ in the learning rule follow an Arcsine distribution of probability (as $\phi_k - \phi_j$ follows a uniform distribution in the interval $[0, 2\pi]$). We confirm this theoretical result with simulation using a Kolmogorov–Smirnov (KS) test. To do that, we consider that both cosine terms are random variables. We denote these variables $X^0_{j,k} = \cos(\phi^0_j - \phi^0_k)$ and $X^\beta_{j,k} = \cos(\phi^\beta_j - \phi^\beta_k)$. The null hypothesis of the test is that both $X^0_{j,k}$ and $X^\beta_{j,k}$ follow a Arcsine law. After generating 1000 samples of $X^0_{j,k}$ and $X^\beta_{j,k}$, the statistical test finally indicates that for both $X^0_{j,k}$ and $X^\beta_{j,k}$, it is impossible to reject the null hypothesis at level $\alpha = 0.05$ (common level for this kind of test).

Figure A4 shows the cumulative distribution function (CDF) of the Arcsine law (dashed red line) and both empirical distribution functions (EDFs) of the random variables $X^0_{j,k}$ (red line) and $X^\beta_{j,k}$ (green line). To obtain this plot, we simulate a one hidden layer network with 50 hidden oscillators and train only one synapse for 1000 iterations. We added the obtained P-values of the KS test to the legend of the plot. It supports the fact that both $X^0_{j,k}$ and $X^\beta_{j,k}$ follow an Arcsine law of probability. Therefore, the coupling update $\Delta\Omega_{j,k}$ follows a difference of two Arcsine distribution, renormalized by the parameter β, i.e. $\Delta\Omega_{j,k} = \frac{1}{\beta}(X^0_{j,k} - X^\beta_{j,k})$ with $X^0_{j,k}, X^\beta_{j,k} \sim \text{Arcsin}(-1, 1)$.

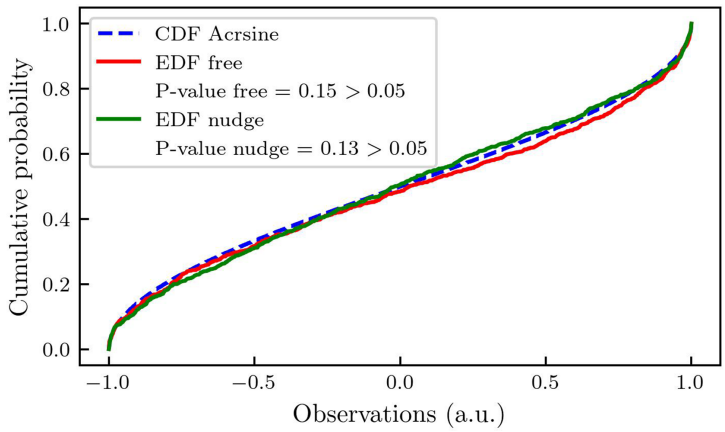

Figure A4. Cumulative distribution function (CDF) of the Arcsine law (dashed red line) and both empirical distribution functions (EDF) of the random variables $X^0_{j,k} = \cos(\phi^0_j - \phi^0_k)$ (red line) and $X^\beta_{j,k} = \cos(\phi^\beta_j - \phi^\beta_k)$ (green line) for a one hidden layer network with 50 hidden oscillators. We used $\beta = 0.05$ GHz and $lr = 0.001$ and 1000 observations of $X^0_{j,k}$ and $X^\beta_{j,k}$.

From this point, we deduce theoretical estimates of the mean and variance of the distribution of $\Delta\Omega_{j,k}$:

$$E\left[\Delta\Omega_{j,k}\right] = 0$$
$$V\left[\Delta\Omega_{j,k}\right] = \frac{1}{\beta^2}.$$

Thus, the variance of the update diverges as we decrease the 'nudging factor' β, leading to extreme values of $\Delta\Omega_{j,k}$. At the same time, the theory of Equilibrium Propagation, described in [27] assures that, when β goes to 0, the weight update $\Delta\Omega_{j,k}$ converges towards the gradient of the loss function $\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\Omega_{j,k}}$, which is a finite value (since our loss function is differentiable). Therefore, by using sufficiently 'small' values of β (which in practice has never appeared to be a issue), we obtain a situation where the majority of the random non-synchronized weight updates are larger in amplitude than the deterministic synchronized weight updates. This explains why the amplitude of the updates is greater than in the synchronized case, as shown empirically in figure A3.

A.7. Equations of dynamics for a network of nonlinear oscillators

In this section we give more details of the dynamics equations for a network of K oscillators following the nonlinear model proposed in [36], and on the impact of several physical parameters.

A.7.1. The nonlinear oscillator model in a large array of oscillators

We consider a system of K coupled oscillators defined by the complex variable c_j, with power p_j and phase ϕ_j influenced by an external drive:

$$\frac{\mathrm{d}c_j}{\mathrm{d}t} + i\omega_j\left(p_j\right)c_j + \left(\Gamma_+\left(p_j\right) - \Gamma_-\left(p_j\right)\right)c_j = \sum_{k=0}^{K}\Omega_{k,j}\mathrm{e}^{\mathrm{i}\beta_{k,j}}c_k + F_j\mathrm{e}^{\mathrm{i}\psi_j(t)} \tag{A1}$$

where:

$c_j = \sqrt{p_j}\mathrm{e}^{\mathrm{i}\phi_j}$: state of oscillator j
$\omega_j(p) = \omega_{0,j} + Np$: frequency of oscillator j
$\Gamma_+(p) = \Gamma_G(1 + Qp)$: positive damping
$\Gamma_-(p) = \sigma I(1 - p)$: negative damping
$\Omega_{k,j}\mathrm{e}^{\mathrm{i}\beta_{k,j}}$: coupling term between oscillators k and j
$F_j\mathrm{e}^{\mathrm{i}\psi_j(t)}$: external drive (bias) applied to oscillator j

A.7.2. Deriving power and phase dynamic equations

From the previous equation in the complex domain, we can derive the coupled system of equations for the power and phase of each oscillator:

$$\begin{aligned}\frac{\mathrm{d}c_j(t)}{\mathrm{d}t} &= \frac{\mathrm{d}}{\mathrm{d}t}\sqrt{p_j(t)}\mathrm{e}^{\mathrm{i}\phi_j(t)} \\ &= \frac{1}{2\sqrt{p_j}}\frac{\mathrm{d}p_j}{\mathrm{d}t}\mathrm{e}^{\mathrm{i}\phi_j} + i\sqrt{p_j}\frac{\mathrm{d}\phi_j}{\mathrm{d}t}\mathrm{e}^{\mathrm{i}\phi_j}.\end{aligned}$$

Then dividing by $\mathrm{e}^{\mathrm{i}\phi_j}$ in both terms,

$$\begin{aligned}&\frac{1}{2\sqrt{p_j}}\frac{\mathrm{d}p_j}{\mathrm{d}t} + i\sqrt{p_j}\frac{\mathrm{d}\phi_j}{\mathrm{d}t} + i\omega_j(p_j)\sqrt{p_j} + \left(\Gamma_+(p_j) - \Gamma_-(p_j)\right)\sqrt{p_j} \\ &= \sum_{k=1}^{K}\Omega_{k,j}\sqrt{p_k}\mathrm{e}^{\mathrm{i}\left(\phi_k-\phi_j+\beta_{k,j}\right)} + F_j\mathrm{e}^{\mathrm{i}\left(\psi_j-\phi_j\right)}.\end{aligned}$$

By taking real and imaginary parts and isolating the derivative terms we obtain,

$$\begin{aligned}\frac{\mathrm{d}p_j}{\mathrm{d}t} + 2p_j\left(\Gamma_+(p_j) - \Gamma_-(p_j)\right) &= 2\sum_{k=1}^{K}\Omega_{k,j}\sqrt{p_jp_k}\cos\left(\phi_k-\phi_j+\beta_{k,j}\right) \\ &\quad + 2F_j\sqrt{p_j}\cos\left(\psi_j-\phi_j\right) \\ \frac{\mathrm{d}\phi_j}{\mathrm{d}t} + \omega_j(p_j) &= \sum_{k=1}^{K}\Omega_{k,j}\sqrt{\frac{p_k}{p_j}}\sin\left(\phi_k-\phi_j+\beta_{k,j}\right) + \frac{F_j}{\sqrt{p_j}}\sin\left(\psi_j-\phi_j\right).\end{aligned}$$

In the phase equation, we recognize several terms that appear in the Kuramoto dynamics (see equation (1)) such as the pairwise couplings $\Omega_{i,j}$ and the frequency term ω_j. However, in this case, both of these terms also depend on the power p_j of the considered oscillator. The coupling term is weighted by the ratio of amplitudes of the coupled oscillators. A term proportional to the oscillator power is added to the frequency term. The proportionality constant N is the nonlinear frequency shift, a physical parameter that depends on the type of oscillator we are considering. We also note the presence of coupling phases $\beta_{k,j}$ within the coupling dynamics. This term influences the synchronization of the system and is closely related to the nonlinear frequency shift N in the frequency term $\omega_j(p_j)$ (you can find more information on the links between the physical parameters N and $\beta_{k,j}$ and their impacts in A.7.4).

The power equation is defined by similar physical influences as the pairwise coupling. Moreover, it also depends on a damping term which is the difference between a positive damping term (natural energy dissipation) and a negative damping term (energy supply from the external energy source, i.e. input current). From this term we derive the minimum amount of injected current to make our system operate at a positive power as described in [36].

To use the framework provided by Equilibrium Propagation with the previous system of nonlinear oscillators, we introduce an energy function that derives from the previous dynamics. As for the Kuramoto model, the oscillators are coupled in phase, so we consider an energy E_ϕ that derives from the phase dynamics of equation (7):

$$E_\phi\left(\phi,p,\theta,\psi_{\text{target}},\beta\right) = \sum_{j=1}^{K}\omega_j(p_j)\phi_j - \sum_{j=1}^{K}\sum_{k=1}^{K}\Omega_{k,j}\sqrt{\frac{p_k}{p_j}}\cos\left(\phi_k-\phi_j+\beta_{k,j}\right) - \sum_{j=1}^{K}\frac{F_j}{\sqrt{p_j}}\cos\left(\psi_j-\phi_j\right).$$

To obtain this energy function, we did not take into account the fact that oscillator powers are also evolving during the stabilization of the system. However, we observe in our simulations that oscillator powers converge towards their steady states in a shorter timescale than the phase of the oscillators (this is particularly true when we increase the oscillator input power). Then, this phenomenon makes this approximation relevant.

We can derive the total energy F_ϕ of the system:

$$F_\phi\left(\phi,p,\theta,\psi_{\text{target}},\beta\right) := E_\phi\left(\phi,p,\theta,\psi_{\text{target}}\right) + \beta\mathcal{L}\left(\phi,\psi_{\text{target}}\right).$$

With,

$\phi = (\phi_j)_{(1\leqslant j\leqslant K)}$: oscillator phases

$p = (p_j)_{(1\leqslant j\leqslant K)}$: oscillator powers

$\theta = (\Omega, F, \psi)$: learning parameters
$\psi_{\text{target}} = (\psi_{\text{target,o}})_{(1 \leqslant o \leqslant O)}$: source phases that encode the target
β: nudge factor

We decided to use a loss function related to the one used in the Kuramoto model, but we also consider the influence of the oscillator power in the dynamics. Then, the loss function we use is weighted by the ratio between the amplitude of the target source $\sqrt{p_{\text{target},o}}$ and the amplitude of the corresponding output oscillator $\sqrt{p_o}$:

$$\mathcal{L} := \sum_{o=1}^{O} \sqrt{\frac{p_{\text{target, o}}}{p_o}} \cos\left(\psi_{\text{target,o}} - \phi_o\right).$$

If we note (p^0, ϕ^0) the states of the oscillators after the 'free phase' and (p^β, ϕ^β) the state of the oscillators after the 'nudge phase', we can obtain the learning rule for each learning parameter (Ω, F, ψ) that will be used during training:

$$\Delta\Omega_{k,j} \propto \frac{1}{\beta}\left(\sqrt{\frac{p_k^\beta}{p_j^\beta}} \cos\left(\phi_k^\beta - \phi_j^\beta + \beta_{k,j}\right) - \sqrt{\frac{p_k^0}{p_j^0}} \cos\left(\phi_k^0 - \phi_j^0 + \beta_{k,j}\right)\right)$$

$$\Delta F_j \propto \frac{1}{\beta}\left(\frac{1}{\sqrt{p_j^\beta}} \cos\left(\psi_j - \phi_j^\beta\right) - \frac{1}{\sqrt{p_j^0}} \cos\left(\psi_j - \phi_j^0\right)\right)$$

$$\Delta\psi_j \propto \frac{F_j}{\beta}\left(\frac{1}{\sqrt{p_j^0}} \sin\left(\psi_j - \phi_j^0\right) - \frac{1}{\sqrt{p_j^\beta}} \sin\left(\psi_j - \phi_j^\beta\right)\right).$$

A.7.3. Equations for a one layer network

Now we give the full equations for a one hidden layer network:

$$\begin{aligned}
\frac{\mathrm{d}p_h^{\text{hid}}}{\mathrm{d}t} + 2p_h^{\text{hid}}\left(\Gamma_+\left(p_h^{\text{hid}}\right) - \Gamma_-\left(p_h^{\text{hid}}\right)\right) &= 2\sum_{i=1}^{I} \Omega_{h,i}^0 \sqrt{p_h^{\text{hid}} p_i^{\text{inp}}} \cos\left(\phi_i^{\text{inp}} - \phi_h^{\text{hid}} + \beta_{i,h}\right) \\
&\quad + 2\sum_{o=1}^{O} \Omega_{o,h}^1 \sqrt{p_h^{\text{hid}} p_o^{\text{out}}} \cos\left(\phi_o^{\text{out}} - \phi_h^{\text{hid}} + \beta_{o,h}\right) \\
&\quad + 2{F^0}_h \sqrt{p_h^{\text{hid}}} \cos\left(\psi_h^0 - \phi_h^{\text{hid}}\right), \quad \forall h \in [\![1, H]\!] \\
\frac{\mathrm{d}\phi_h^{\text{hid}}}{\mathrm{d}t} + \omega_h\left(p_h^{\text{hid}}\right) &= \sum_{i=1}^{I} \Omega_{h,i}^0 \sqrt{\frac{p_i^{\text{inp}}}{p_h^{\text{hid}}}} \sin\left(\phi_i^{\text{inp}} - \phi_h^{\text{hid}} + \beta_{i,h}\right) \\
&\quad + \sum_{o=1}^{O} \Omega_{o,h}^1 \sqrt{\frac{p_o^{\text{out}}}{p_h^{\text{hid}}}} \sin\left(\phi_o^{\text{out}} - \phi_h^{\text{hid}} + \beta_{o,h}\right) \\
&\quad + \frac{{F^0}_h}{\sqrt{p_h^{\text{hid}}}} \sin\left(\psi_h^0 - \phi_h^{\text{hid}}\right), \quad \forall h \in [\![1, H]\!] \\
\frac{\mathrm{d}p_o^{\text{out}}}{\mathrm{d}t} + 2p_o^{\text{out}}\left(\Gamma_+\left(p_o^{\text{out}}\right) - \Gamma_-\left(p_o^{\text{out}}\right)\right) &= 2\sum_{h=1}^{H} \Omega_{o,h}^1 \sqrt{p_h^{\text{out}} p_i^{\text{hid}}} \cos\left(\phi_h^{\text{hid}} - \phi_o^{\text{out}} + \beta_{h,o}\right) \\
&\quad + 2{F^1}_o \sqrt{p_o^{\text{out}}} \cos\left(\psi_o^1 - \phi_o^{\text{out}}\right), \quad \forall o \in [\![1, O]\!] \\
\frac{\mathrm{d}\phi_o^{\text{out}}}{\mathrm{d}t} + \omega_o\left(p_o^{\text{out}}\right) &= \sum_{h=1}^{H} \Omega_{o,h}^1 \sqrt{\frac{p_h^{\text{hid}}}{p_o^{\text{out}}}} \sin\left(\phi_h^{\text{hid}} - \phi_o^{\text{out}} + \beta_{h,o}\right) \\
&\quad + \frac{{F^1}_o}{\sqrt{p_o^{\text{out}}}} \sin\left(\psi_o^1 - \phi_o^{\text{out}}\right), \quad \forall o \in [\![1, O]\!]
\end{aligned}$$

where:

(I, H, O): number of oscillators (or sources) in each layer, respectively input, hidden and output layers

$((p_i^{\text{inp}})_{(1\leqslant i\leqslant I)}, (p_h^{\text{hid}})_{(1\leqslant h\leqslant H)}, (p_o^{\text{out}})_{(1\leqslant o\leqslant O)})$: oscillator power (sources for the input), respectively in the input, hidden and output layers

$((\phi_i^{\text{inp}})_{(1\leqslant i\leqslant I)}, (\phi_h^{\text{hid}})_{(1\leqslant h\leqslant H)}, (\phi_o^{\text{out}})_{(1\leqslant o\leqslant O)})$: oscillator phases in the stationary frame (sources for the input), respectively in the input, hidden and output layers

$\Omega^0 = (\Omega_{h,i})_{(1\leqslant h\leqslant H),(1\leqslant i\leqslant I)}$: coupling matrix between input and hidden layers

$\Omega^1 = (\Omega_{o,h})_{(1\leqslant o\leqslant O),(1\leqslant h\leqslant H)}$: coupling matrix between hidden and output layers

$(F^0, \psi^0) = ({F^0}_h, \psi_h^0)_{(1\leqslant h\leqslant H)}$: external drives (amplitude and phase) applied to oscillators in the hidden layer

$(F^1, \psi_{\text{ext}}^1) = ({F^1}_o, \psi_{\text{ext,o}}^1)_{(1\leqslant o\leqslant O)}$: external drives (amplitude and phase) applied to oscillators in the output layer

A.7.4. Nonlinear frequency shift N and coupling phase $\beta_{k,j}$

A very particular aspect of the nonlinear oscillator model of oscillators is the dependency between an oscillator frequency and its power. In fact, a term proportional to the oscillator power is added to the natural frequency of the oscillator. The standard frequency term ω_0 in 1 becomes $\omega_0 + Np$ with p the oscillator power and N the nonlinear frequency shift, a intrinsic parameter of the considered oscillator. In the theoretical framework exposed in [36], the authors showed in case of two coupled oscillators that this parameter has a direct impact on the size of the synchronization region of the system.

To achieve a maximum size of the synchronization bandwidth, authors in [36] also explained that we need to choose carefully the coupling phases $(\beta_{k,j})_{1\leqslant k,j\leqslant K}$ that appears in the coupling dynamics. Physically, the value of these parameters depends on the coupling mechanism or the time delay between the oscillators. For two almost identical nonlinear oscillators, they showed that a careful choice of coupling phases can increase the size of the synchronization bandwidth by compensating for the frequency shift of the oscillators. In that specific case, we would need to choose the coupling phases as:

$$\beta_{k,j} = \beta = \arctan(\nu) = \arctan\left(\frac{N}{\Gamma_G(Q + I_{\text{ratio}})}\right). \tag{A2}$$

To our knowledge, there is no analytic relation between the coupling phases and the size of the locking bandwidth in the case of an arbitrary number of such oscillators with random pairwise couplings $\Omega_{k,j}$. For the Kuramoto model, it has been shown in [42] that in some particular conditions, it is possible to obtain an analytic expression for the number of synchronized oscillators as a function of the coupling phases. But these results do not clearly show that the synchronization is maximum for a particular value of the coupling phases. If we use the value of equation (A2) in the case of a large network of oscillators, it appears that the nonlinear model can be reduced to the fully-synchronized Kuramoto model. With this result, we can provide a fair comparison between both models as showed in table A1.

As mentioned in [36], by introducing the effective oscillator phases, $\tilde{\phi}_j$ that depends on the nonlinear frequency shift and the oscillator power, the nonlinear oscillator model can be reduced to the following system of equations:

$$\frac{\mathrm{d}\tilde{\phi}_j}{\mathrm{d}t} + \omega_{g,j} = \sum_{k=1}^{K} \tilde{\Omega}_{k,j} \sin\left(\tilde{\phi}_k - \tilde{\phi}_j + \tilde{\beta}_{k,j}\right), \quad \forall j \in [\![1, K]\!] \tag{A3}$$

with $\omega_{g,j}$ the free-running frequency of the jth oscillator, $\tilde{\Omega}_{k,j}$ and $\tilde{\beta}_{k,j}$ the renormalized couplings frequencies and phases influenced by the nonlinear frequency shift.

Equation (A3) corresponds to the Kuramoto model with the coupling phases $\tilde{\beta}_{k,j} = \beta_{k,j} - \arctan(N/(\Gamma_G(Q + I_{\text{ratio}})))$. Then, using equation (A2), we obtain a renormalized coupling phases equal to 0. Hence equation (A3) corresponds to the Kuramoto model described in section 2.1.

A.8. Simulations details

In this section, we provide several information related to the simulations carried out in this work.

A.8.1. Physical framework

All the values of the physical quantities used in this work (frequency, couplings, damping constant) are inspired from the field of spintronics [43]. In particular, we consider sources that emit at a frequency $\omega_0 = 4.2$ GHz, the couplings between oscillators and the 'nudge factor' are of the order of a hundred MHz.

A.8.2. Simulation framework

All the simulations have been carried out using the Pytorch framework of Python. The largest simulations of both models (Kuramoto and nonlinear models) have been run on two to four GPUs V100 operating in

parallel. To solve the dynamics equations (which consumed most of the resources and time), we used second-order Runge–Kutta numerical methods.

A.8.3. Simulation issues

The results on MNIST with a network of oscillators following the nonlinear universal model were quite challenging to obtain since both the physical system and the Equilibrium Propagation algorithm require high computational power. We also encountered several simulation issues. In fact, the simulated model is composed of two coupled equations (power and phase) with the power variable that needs to remain positive. In case of a large number of oscillators, we observed that several of them may collapse instead of reaching their positive final value. This is due to the coupling interactions between oscillators which can greatly affect their behaviors. This has often led to the crash of the simulations. To prevent this phenomenon, we increase the amount of the current injected into the oscillators. For a high quantity of injected current, we observe a more stable power dynamics which directly implies an increase of the final accuracy.

Simulations with the Kuramoto model were easier to handle. In that case, oscillators are only described with one variable (their phase). Then for small networks (a few dozen of coupled oscillators), it was possible to carry out training on a CPU. However for larger network we were forced to increase precision when solving the equations of dynamics, which required the use of GPUs.

A.8.4. Database

All conducted simulations were carried out on the MNIST database [41] and its rescaled version provided by the scikit-learn package of Python. The original database is divided into training and test sets, that are respectively composed of 60 000 and 10 000 28×28 images of greyscale handwritten digits. The rescaled database is composed of one set of 1797 8×8 images (https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html).

A.8.5. Choice of hyperparameters

The description of the hyperparameters used in the conducted simulations is given in table A1. To choose the hyperparameters of Equilibrium Propagation (N_{free}, N_{nudge}, ϵ and β), we tried to understand which combination of these parameters give a good quality gradient approximation. In fact in [38], the authors demonstrate theoretically and experimentally that gradients computed by Equilibrium Propagation tend to approximate those calculated by the back-propagation through time algorithm (BPTT) for vanishing values of the 'nudge factor' β. Hence, we compared the gradients produced by Equilibrium Propagation and those by BPTT for different hyperparameters.

Figure A5 shows the evolution of the cosine similarity between the gradients computed by EP (P-EP and C-EP denote respectively Positive-EP and Centered-EP, denomination used in [26]) and those computed by BPTT for different values of the 'nudge factor' β. As expected, the metric is maximal in both layer for very small values of β. As we increase its value, the cosine similarity decreases, which means that the direction of the computed gradients deviates from the one computed by BPTT. This result has been confirmed through simulations, i.e. 'large' values of β usually lead to poor performance of the model. As proposed in figure A5, we decided to choose $\beta = 10$ MHz, a relatively small value of the nudge factor, to provide good gradient approximations. We observe through our simulations that smaller values of β does not really increase the performances of the models.

For the other parameters (N_{free}, N_{nudge} and ϵ), we were facing a trade-off between the quality of approximation and the computation time. To obtain better approximations, we should increase N_{free} and N_{nudge}, and decrease ϵ. However, this also increases computation times because the relaxation of the oscillator is slower. In this situation, we observe that the quality of the gradients increases after a few iterations. This can be explained by the fact that initially the weights of the network are random, so the order of magnitude of the computed gradients is larger than after a few iterations, once the weights have been updated in the right direction. It implies that the precision used to solve the system dynamics can be a little bit reduced. After several simulations, we choose the parameters given in table A1. Regarding the number of epochs, we have been constraint by the computation time required by the nonlinear oscillator model. In fact, this model involves two coupled variables (power and phase of an oscillator), so its computation time is greater than the Kuramoto model (that involves only one variable). We were able to train this model with sufficient precision only for 40 epochs (and 100 epochs for Kuramoto). Finally, the choice of the learning rates was relatively straightforward for both models by observing the error and loss curves.

Table A1. Hyperparameters used for training. 'Hidden' indicates the number of hidden units, N_{free} and N_{nudge} correspond respectively to the number of steps to solve the dynamics of the system for the 'free' and 'nudge' phases, with the resolution step ϵ, and β denotes the value of the nudge factor.

Model	Hidden	N_{free}	N_{nudge}	ϵ	β ($\times 10^2$ *MHz*)	Epochs	Learning rates
Kuramoto	500	1500	1000	0.01	0.1	100	0.001–0.0005
Non-linear	500	1500	1000	0.01	0.1	40	0.0005-0.000 25

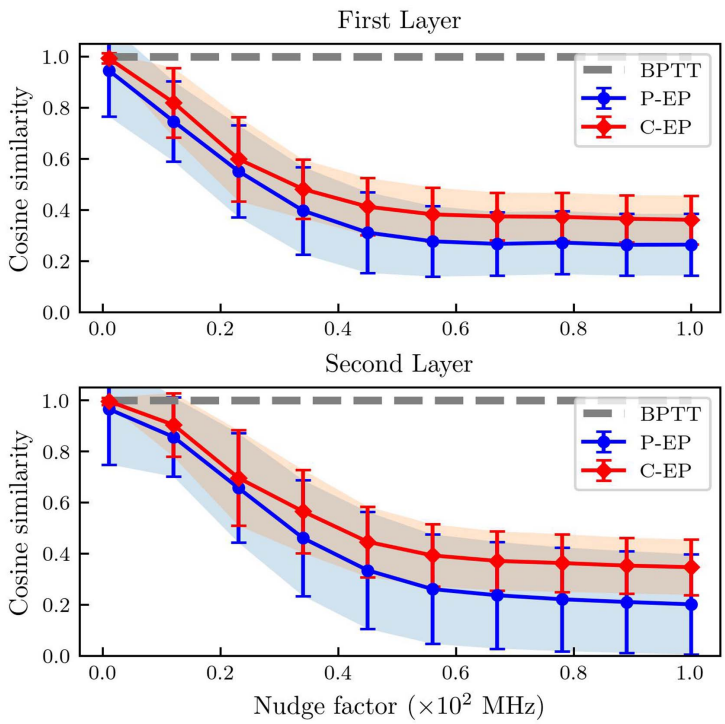

Figure A5. Average cosine similarity between gradients produced by EP (P-EP and C-EP denote respectively Positive-EP and Centered-EP, denomination used in [26]) and BPTT as a function of the nudge factor β, for a Kuramoto model with $N_{free} = 2000$, $N_{nudge} = 500$, $\epsilon = 0.2$. The average is done over 200 random images of MNIST database.

A.8.6. Optimization

Model optimization was performed using Adam optimizer [40] with batches of size 64 and a learning rate scheduler that multiply the current learning rates by 0.98 at the end of each training epoch. Moreover we set the learning rate of the bias phase to 100 times the one of the bias amplitude.

A.9. Impact of the bias on the learning

We tried to quantify the impact of the external drives applied to oscillators (called bias in Machine Learning). In the perspective of a hardware implementation, this question can be relevant because it changes the total number of parameters in the system. To do that, we compare the performance of two models of fully-synchronized Kuramoto oscillators, one including external drives and the other not. We compare these models on Digits and MNIST databases. For Digits database (8×8 pixels per image), we use a one-hidden layer network with 50 hidden neurons. For MNIST (28×28 pixels per image), we use 500 hidden neurons.

Figure A6 shows the evolution of the training loss function plotted in logarithmic scale for 100 epochs for Digits database (left) and MNIST database (right). In both cases, we observe that models with bias slightly outperform models without bias. Moreover, for Digits, we also note that the standard deviation of the final obtained state is higher if there is no bias in the network, which means that training a network with bias increases its stability and reproducibility.

Therefore, it seems that the impact of an external drive is not very significant in large networks. Compared to fully-connected layers, bias represent around 0.2%–0.3% (for MNIST) of the total number of parameters. In smaller networks such as the one we trained on Digits, the gain in performance is a bit more significant, and the stability of training is increased. This could be interesting if we want to train a network in the presence of noise for example.

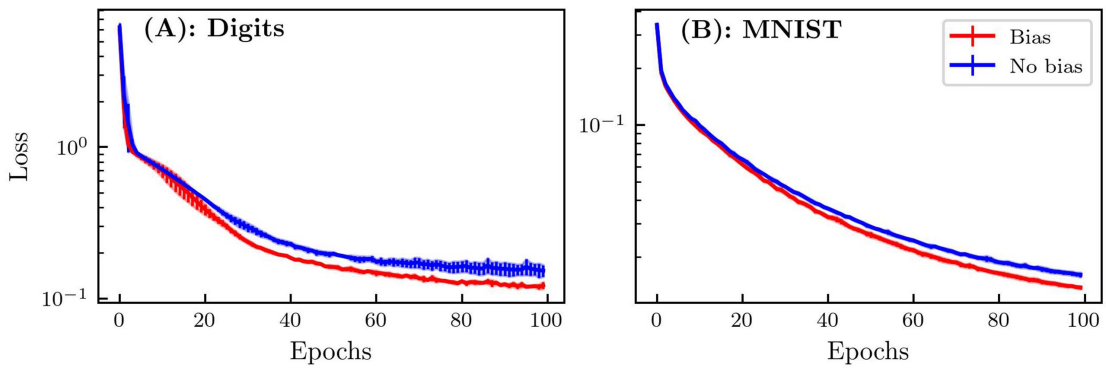

Figure A6. Average loss curves in logarithmic scale (mean over 3 runs) for a Kuramoto network trained on Digits database (left) and MNIST database (right) with and without external drives (bias).

ORCID iDs

Théophile Rageau 0009-0000-0881-2917
Julie Grollier 0000-0003-4866-4490

References

[1] Todri-Sanial A, Delacour C, Abernot M and Sabo F 2024 Computing with oscillators from theoretical underpinnings to applications and demonstrators *npj Unconv. Comput.* **1** 1–16
[2] Wang T and Roychowdhury J 2019 OIM: oscillator-based Ising machines for solving combinatorial optimisation problems *Unconventional Computation and Natural Computation: 18th Int. Conf., UCNC 2019*, (*Proc. 18*) (*Tokyo, Japan, 3–7 June 2019*), (Springer) pp 232–56
[3] Moy W, Ahmed I, Chiu P-W, Moy J, Sapatnekar S S and Kim C H 2022 A 1,968-node coupled ring oscillator circuit for combinatorial optimization problem solving *Nat. Electron.* **5** 310–7
[4] Bazzi A, Hardy E, Ballester J, Badets F and Hutin L 2024 Optimizing with phases: design and application space assessment for networks of phase-locked ring oscillators *2024 IEEE 6th Int. Conf. on AI Circuits and Systems (AICAS)* (IEEE) pp 602–6
[5] English L Q, Zampetaki A, Kalinin K, Berloff N and Kevrekidis P G 2022 An Ising machine based on networks of subharmonic electrical resonators *Commun. Phys.* **5** 333
[6] Chou J, Bramhavar S, Ghosh S and Herzog W 2019 Analog coupled oscillator based weighted Ising machine *Sci. Rep.* **9** 14786
[7] Delacour C, Carapezzi S, Boschetto G, Abernot M, Gil T, Azemard N and Todri-Sanial A 2023 A mixed-signal oscillatory neural network for scalable analog computations in phase domain *Neuromorph. Comput. Eng.* **3** 034004
[8] Graber M and Hofmann K 2024 An integrated coupled oscillator network to solve optimization problems *Commun. Eng.* **3** 116
[9] Hölzel R 2013 A neural network of weakly coupled nonlinear oscillators with a global, time-dependent coupling-theory and experiment *PhD Dissertation* Technische Universität München
[10] Nikonov D E, Kurahashi P, Ayers J S, Li H, Kamgaing T, Dogiamis G C, Lee H-J, Fan Y and Young I 2020 Convolution inference via synchronization of a coupled CMOS oscillator array *IEEE J. Exploratory Solid State Comput. Dev. Circuits* **6** 170–6
[11] Choi W, van Bodegraven T, Verest J, Maher O, Falcone D F, Porta A L, Jubin D, Offrein B J, Karg S, Bragaglia V and Todri-Sanial A 2025 Hardware implementation of ring oscillator networks coupled by BEOL integrated ReRAM for associative memory tasks (arXiv:2503.14126)
[12] Torrejon J *et al* 2017 Neuromorphic computing with nanoscale spintronic oscillators *Nature* **547** 428–31
[13] Romera M *et al* 2018 Vowel recognition with four coupled spin-torque nano-oscillators *Nature* **563** 230–4
[14] Buzsaki G 2006 *Rhythms of the Brain* (Oxford university press)
[15] Strogatz S H and Stewart I 1993 Coupled oscillators and biological synchronization *Sci. Am.* **269** 102–9
[16] Lehnertz K, Bialonski S, Horstmann M-T, Krug D, Rothkegel A, Staniek M and Wagner T 2009 Synchronization phenomena in human epileptic brain networks *J. Neurosci. Methods* **183** 42–48
[17] Malagarriga D, García-Vellisca M A, Villa A E, Buldú J M, García-Ojalvo J and Pons A J 2015 Synchronization-based computation through networks of coupled oscillators *Front. Comput. Neurosci.* **9** 97
[18] Hoppensteadt F C and Izhikevich E M 1999 Oscillatory neurocomputers with dynamic connectivity *Phys. Rev. Lett.* **82** 2983
[19] Kuramoto Y 1975 Self-entrainment of a population of coupled non-linear oscillators *Int. Symp. on Mathematical Problems in Theoretical Physics: (23–29 January 1975)* (Kyoto University, Springer) pp 420–2
[20] Popescu B, Csaba G, Popescu D, Fallahpour A H, Lugli P, Porod W and Becherer M 2018 Simulation of coupled spin torque oscillators for pattern recognition *J. Appl. Phys.* **124** 152128
[21] Rudner T, Porod W and Csaba G 2024 Design of oscillatory neural networks by machine learning *Front. Neurosci.* **18** 1307525
[22] Follmann R, Macau E, Rosa E and Piqueira J 2014 Phase oscillatory network and visual pattern recognition *IEEE Trans. Neural Netw. Learn. Syst.* **26** 08
[23] Rumelhart D E, Hinton G E and Williams R J 1986 Learning representations by back-propagating errors *Nature* **323** 533–6
[24] Werbos P J 1990 Backpropagation through time: what it does and how to do it *Proc. IEEE* **78** 1550–60
[25] Kuninti S and Rooban S 2021 Backpropagation algorithm and its hardware implementations: A review *J. Phys.: Conf. Ser.* **1804** 012169
[26] Scellier B, Ernoult M, Kendall J and Kumar S 2024 Energy-based learning algorithms for analog computing: a comparative study *Advances in Neural Information Processing Systems* vol 36

[27] Scellier B and Bengio Y 2017 Equilibrium propagation: bridging the gap between energy-based models and backpropagation *Front. Comput. Neurosci.* **11** 24
[28] Yi S-I, Kendall J D, Williams R S and Kumar S 2023 Activity-difference training of deep neural networks using memristor crossbars *Nat. Electron.* **6** 45–51
[29] Dillavou S, Beyer B D, Stern M, Liu A J, Miskin M Z and Durian D J 2024 Machine learning without a processor: emergent learning in a nonlinear analog network *Proc. Natl Acad. Sci.* **121** e2319718121
[30] Laydevant J, Marković D and Grollier J 2024 Training an Ising machine with equilibrium propagation *Nat. Commun.* **15** 3671
[31] Nest T and Ernoult M 2024 Towards training digitally-tied analog blocks via hybrid gradient computation (arXiv:2409.03306)
[32] Wang Q, Wanjura C C and Marquardt F 2024 Training coupled phase oscillators as a neuromorphic platform using equilibrium propagation (arXiv:2402.08579)
[33] Jenkins A S, Martins L, Benetti L C, Schulman A, Anacleto P, Claro M S, Caha I, Deepak F L, Paz E and Ferreira R 2024 The impact of local pinning sites in magnetic tunnel junctions with non-homogeneous free layers *Commun. Mater.* **5** 7
[34] Tsunegi S, Taniguchi T, Lebrun R, Yakushiji K, Cros V, Grollier J, Fukushima A, Yuasa S and Kubota H 2018 Scaling up electrically synchronized spin torque oscillator networks *Sci. Rep.* **8** 13475
[35] Rodrigues F A, Peron T K D, Ji P and Kurths J 2016 The kuramoto model in complex networks *Phys. Rep.* **610** 1–98
[36] Slavin A and Tiberkevich V 2009 Nonlinear auto-oscillator theory of microwave generation by spin-polarized current *IEEE Trans. Magn.* **45** 1875–918
[37] Laborieux A, Ernoult M, Scellier B, Bengio Y, Grollier J and Querlioz D 2021 Scaling equilibrium propagation to deep convnets by drastically reducing its gradient estimator bias *Front. Neurosci.* **15** 633674
[38] Ernoult M, Grollier J, Querlioz D, Bengio Y and Scellier B 2019 Updates of equilibrium prop match gradients of backprop through time in an RNN with static input *Advances in Neural Information Processing Systems* vol 32
[39] Ruder S 2017 An overview of gradient descent optimization algorithms (arXiv:1609.04747)
[40] Kingma D P 2014 Adam: a method for stochastic optimization (arXiv:1412.6980)
[41] LeCun Y, Bottou L, Bengio Y and Haffner P 1998 Gradient-based learning applied to document recognition *Proc. IEEE* **86** 2278–324
[42] Sakaguchi H and Kuramoto Y 1986 A soluble active rotater model showing phase transitions via mutual entertainment *Prog. Theor. Phys.* **76** 576–81
[43] Albertsson D I, Zahedinejad M, Houshang A, Khymyn R, Åkerman J and Rusu A 2021 Ultrafast Ising machines using spin torque nano-oscillators *Appl. Phys. Lett.* **118** 112404